

Technology Resources

BarkahFlow — Technology Resources & Stack Documentation

This document outlines the core technologies, libraries, and frameworks required to build BarkahFlow. Our architecture relies on a Single Frontend, Dual-Native-Bridge design pattern. This means we write our UI once, and deploy it locally to both Desktop and Mobile environments using native hardware wrappers.

1. Core UI & Application Framework

Our frontend logic and user interface are built on a modern, React-based foundation. Because our app relies on local SQLite databases on the user's device, we do not use server-side rendering (SSR).

- Next.js
 - Resource: <https://nextjs.org/>
 - Role in Project: Serves as our core React framework.
 -
 - Implementation Note: The application is compiled entirely as a Static Single Page Application (SPA) utilizing Next.js's static export capabilities (`output: 'export'`). This allows us to bundle the HTML/CSS/JS directly into our native wrappers without needing a Node.js server to run locally.

2. Native Runtime Wrappers (The Bridge)

To achieve our offline-first architecture, the compiled Next.js frontend is injected into native shells. These wrappers provide the "bridge" to the device's native SQLite engine.

- Tauri v2 (Desktop Target)
 - Resource: <https://v2.tauri.app/>
 -
 - Role in Project: The Desktop Target Environment (Rust backend container using the system's native WebView2/WebKit rendering layer).
 -
 - Implementation Note: Tauri will mount `tauri-plugin-sql`, executing raw queries through a native Rust driver directly to `libsqlite3`. This ensures ultra-fast, offline POS checkouts on Windows, macOS, and Linux hardware.
- Capacitor (Mobile Target)
 - Resource: <https://capacitorjs.com/>
 -
 - Role in Project: The Mobile Target Environment, wrapping the client bundle into optimized native

Android and iOS application shells.

-
- Implementation Note: Capacitor will mount `@capacitor-community/sqlite`, executing raw queries via native Java/Kotlin or Swift persistence pipelines. This allows merchants to use BarkahFlow on tablets and smartphones completely offline.

3. UI Component Architecture & Design System

BarkahFlow requires a highly readable, Arabic-first interface. These libraries provide accessible, customizable building blocks that natively support our RTL (Right-to-Left) requirements.

- Shadcn UI
 - Resource: <https://ui.shadcn.com/>
 -
 - Role in Project: Provides our base UI layer using Radix-based headless components.
 -
 - Implementation Note: We chose this because it includes native, system-wide Left-to-Right (LTR) and Right-to-Left (RTL) localization to support fluent Arabic bookkeeping. Since it is headless, it gives us complete control over the BarkahFlow styling (Deep Emerald Teal Slate and Fresh Mint accents).
- ReUI Components
 - Resource: <https://reui.io/components>
 - Role in Project: Supplemental UI components to accelerate frontend development.
 - Implementation Note: Use these for complex, pre-built interactive elements (like data tables, modals, or advanced form inputs) that would otherwise take days to build from scratch. Ensure all components used are vetted for RTL layout compatibility.
- Efferd Blocks
 - Resource: <https://efferd.com/blocks>
 - Role in Project: Pre-designed section blocks (e.g., dashboard layouts, stat cards, pricing/subscription gateways).
 - Implementation Note: Highly useful for rapidly prototyping our reporting dashboards and the initial Supabase subscription gateway block. These blocks must be customized to match the BarkahFlow brand identity.